

ADAPT-ECG — Documentación Técnica Completa

Sistema inteligente con reentrenamiento continuo para la detección adaptativa de patologías cardiovasculares basado en señales ECG

Residencia Profesional — Ingeniería Biomédica

Instituto Tecnológico de Tijuana

Autor: Elias Bejarano Lozada (#20213057)

1. ¿Qué es ADAPT-ECG?

ADAPT-ECG es un sistema de clasificación automática de arritmias cardíacas a partir de señales ECG. Utiliza una Red Neuronal Convolucional (CNN) entrenada con la base de datos MIT-BIH Arrhythmia Database y es capaz de **reentrenarse de forma incremental** con nuevos datos sin olvidar lo aprendido anteriormente, gracias a un mecanismo llamado **Replay Buffer**.

El sistema clasifica cada latido en una de las **5 clases del estándar AAMI** (Association for the Advancement of Medical Instrumentation):

Clase	Nombre completo	Significado clínico
N	Normal	Latido sinusal normal
S	Supraventricular	Arritmia originada en las aurículas
V	Ventricular	Arritmia originada en los ventrículos (peligrosa)
F	Fusión	Latido híbrido entre Normal y Ventricular
Q	Desconocido	Artefacto, marcapasos o no clasificable

2. Base de Datos: MIT-BIH Arrhythmia Database

- **Fuente:** PhysioNet (Moody & Mark, 2001)
- **Registros:** 48 grabaciones de ECG ambulatorio
- **Duración:** ~30 minutos por registro
- **Frecuencia de muestreo:** 360 Hz
- **Canales:** 2 (se usa el canal MLII — derivación estándar)
- **Anotaciones:** Cada pico R etiquetado por cardiólogos expertos
- **Total de latidos procesados:** 94,627

Los registros van del 100 al 234 y cubren una amplia variedad de pacientes con distintos tipos de arritmias.

3. Arquitectura del Sistema — 4 Fases

■ FASE 1 – Adquisición y Preprocesamiento de Datos	■
■ Archivos: src/data/ingest.py + src/data/preprocess.py	■
■ Entrada: 48 registros MIT-BIH (.dat, .hea, .atr)	■
■ Salida: data/processed/X.npy + y.npy (94,627 latidos)	■
■ FASE 2 – Entrenamiento del Modelo Base	■
■ Archivo: notebooks/fase2_entrenamiento.ipynb	■
■ Entrada: X.npy, y.npy	■
■ Salida: models/ecg_cnn_base.pth	■
■ FASE 3 – Reentrenamiento Incremental (ADAPT)	■
■ Archivo: notebooks/fase3_reentrenamiento.ipynb	■
■ Entrada: ecg_cnn_base.pth + nuevos registros	■
■ Salida: models/ADAPT-ECG-RETRAINED.pth	■
■ FASE 4 – Evaluación y Comparación	■
■ Archivo: notebooks/fase4_evaluacion.ipynb	■
■ Entrada: ecg_cnn_base.pth vs ADAPT-ECG-RETRAINED.pth	■
■ Salida: models/fase4_resultados.json + gráficas	■
↓ Integración	
src/ui/app.py – Interfaz Streamlit	

4. Descripción de Cada Archivo

4.1 Configuración

src/config/settings.py

Centraliza todos los parámetros del sistema para que ningún archivo tenga valores hardcodeados:

- Rutas del proyecto (DATA_RAW_DIR, MODELS_DIR, etc.)
- Parámetros de la señal: frecuencia de muestreo = 360 Hz, canal MLII
- Parámetros del filtro: Butterworth pasa-banda 0.5–40 Hz, orden 4
- Parámetros de segmentación: ventana de 72 muestras (90ms antes + 110ms después del pico R)
- Mapeo de símbolos MIT-BIH a clases AAMI
- Hiperparámetros de entrenamiento: 30 épocas, batch 64, lr 0.001

4.2 Datos

src/data/ingest.py

Descarga y carga los registros MIT-BIH desde PhysioNet usando la librería wfdb:

- download_record(id) — descarga un registro individual (omite si ya existe)
- download_all() — descarga los 48 registros con barra de progreso
- load_record(id) — lee señal + anotaciones de un registro local
- load_all_records() — carga todos los registros en un diccionario
- record_info(id) — devuelve resumen del registro (duración, cantidad de latidos, distribución de clases)

src/data/preprocess.py

Implementa el pipeline completo de preprocesamiento:

- `bandpass_filter(señal)` — filtro Butterworth zero-phase (sin distorsión de fase)
 - `segment_beats(señal, picos_R, símbolos)` — extrae ventanas de 72 muestras centradas en cada pico R
 - `normalize_beats(latidos)` — normalización z-score independiente por latido
 - `process_record(señal, anotación)` — ejecuta los 3 pasos anteriores en secuencia
 - `process_and_save(registros)` — procesa todos los registros y guarda X.npy e y.npy
 - `load_processed()` — carga el dataset ya procesado
-

4.3 Interfaz

src/ui/app.py

Aplicación Streamlit principal con toda la lógica integrada:

- Definición de la arquitectura CNN (`class ECG_CNN`)
- Implementación del Replay Buffer (`class ReplayBuffer`)
- Función de reentrenamiento incremental (`incremental_train`)
- Carga de modelos (estático y adaptativo)
- Visualización interactiva de la señal ECG con Plotly
- Clasificación en tiempo real latido por latido
- Historial de sesiones de reentrenamiento

Modos de uso:

1. **MIT-BIH:** Selecciona cualquiera de los 48 registros, clasifica, y opcionalmente reentrena
 2. **CSV propio:** Sube tu propia señal ECG, detecta picos R automáticamente con `find_peaks` y clasifica
-

4.4 Modelos guardados

Archivo	Descripción
models/ecg_cnn_base.pth	Pesos del modelo estático (entrenado una sola vez)
models/ecg_cnn_base_meta.json	Metadatos del modelo: arquitectura, hiperparámetros, métricas del entrenamiento
models/ADAPT-ECG-RETRAINED.pth	Pesos del modelo adaptativo (reentrenado 5 veces)
models/replay_buffer.npz	Memoria persistente del buffer de reentrenamiento
models/retrain_history.json	Registro JSON de cada sesión de reentrenamiento
models/fase3_resultados.json	Comparación Estático vs Adaptativo (Fase 3)
models/fase4_resultados.json	Evaluación completa con métricas por clase y test estadístico

4.5 Notebooks

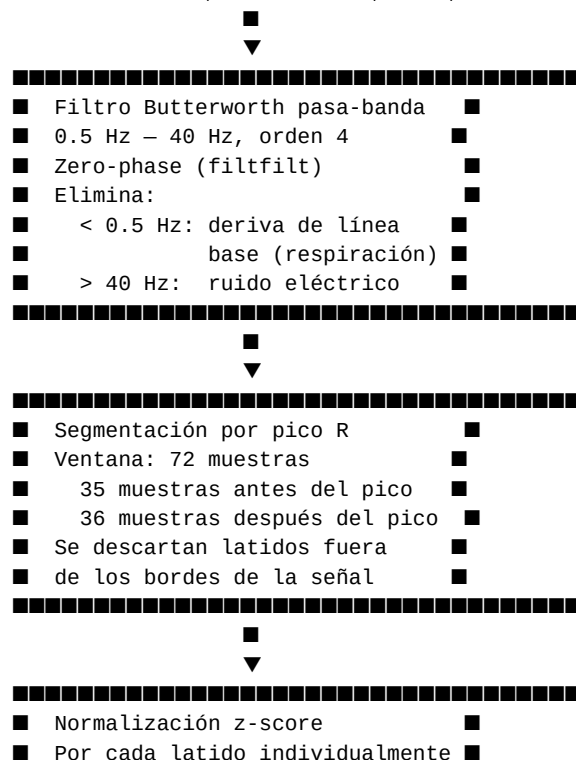
Notebook	Fase	Contenido
notebooks/fase2_entrenamiento.ipynb	2	Entrenamiento de la CNN base, curvas de aprendizaje, métricas de validación
notebooks/fase3_reentrenamiento.ipynb	3	Reentrenamiento incremental con Replay Buffer, comparación antes/después
notebooks/fase4_evaluacion.ipynb	4	Evaluación final completa: confusion matrix, ROC curves, test McNemar

4.6 Scripts y documentos

Archivo	Descripción
scripts/run_phase1.py	Ejecuta descarga + preprocesamiento desde terminal
scripts/retrain.ps1	PowerShell para lanzar reentrenamiento
scripts/train.ps1	PowerShell para lanzar entrenamiento
scripts/run_api.ps1	PowerShell para levantar la app Streamlit
requirements.txt	Dependencias del proyecto
docs/*.png	Gráficas generadas: distribución de latidos, matrices de confusión, curvas ROC, evolución por lotes

5. Pipeline de Preprocesamiento (Fase 1)

Señal ECG cruda – 360 Hz, ~30 minutos, ~650,000 muestras



¿Por qué CNN 1D y no RNN o Transformer?

- La señal ECG tiene **patrones locales** (la forma del complejo QRS) que las convoluciones capturan eficientemente
- Más rápida y ligera que una RNN
- BatchNorm estabiliza el entrenamiento con el desbalanceo de clases
- Dropout previene el sobreajuste

7. Entrenamiento del Modelo Base (Fase 2)

Parámetro	Valor
Dataset total	94,627 latidos
Split entrenamiento	80% (75,701 latidos)
Split validación	20% (18,926 latidos)
Épocas	30
Batch size	64
Optimizador	Adam
Learning rate	0.001
Función de pérdida	CrossEntropyLoss
Semilla aleatoria	42

Métricas del modelo base (ecg_cnn_base.pth):

Métrica	Valor
Accuracy (validación)	93.18%
F1-macro	77.83%
F1-weighted	94.12%

8. Reentrenamiento Incremental — El Núcleo Adaptativo (Fase 3)

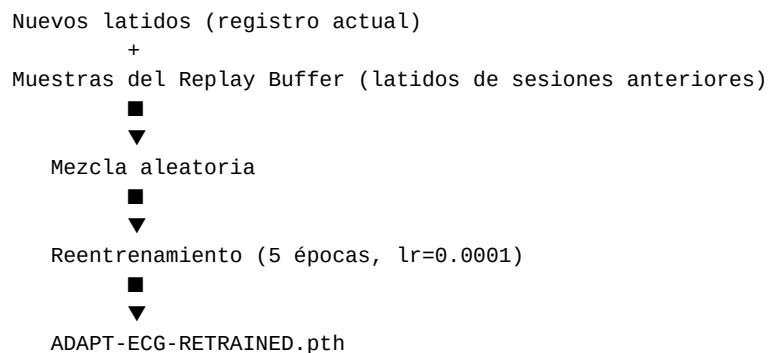
8.1 El Problema: Olvido Catastrófico

Cuando una red neuronal se reentrena con nuevos datos, tiende a **olvidar lo que aprendió antes**. Esto se llama "olvido catastrófico" y es el principal obstáculo del aprendizaje continuo.

Ejemplo: Si el modelo aprendió a clasificar latidos del registro 100 y luego lo reentrenas solo con datos del registro 200, empezará a fallar en el registro 100.

8.2 La Solución: Replay Buffer

El Replay Buffer es una **memoria circular** que almacena latidos de sesiones anteriores. Al reentrenar, mezcla los datos nuevos con muestras del buffer:



Parámetros del Replay Buffer:

Parámetro	Valor
Capacidad máxima	2,000 latidos
Estrategia	FIFO (los más antiguos salen primero)
Proporción replay/nuevos	50% de los nuevos datos se toman del buffer
Épocas por sesión	5
Learning rate reentrenamiento	0.0001 (10x menor que el entrenamiento base)

8.3 Algoritmo Paso a Paso

```
def incremental_train(model, nuevos_beats, nuevas_labels, replay_buffer):  
  
    # Paso 1: Agregar nuevos datos al buffer  
    replay_buffer.add(nuevos_beats, nuevas_labels)  
  
    for época in range(5):  
        # Paso 2: Muestrear del buffer (50% del tamaño de nuevos datos)  
        n_replay = len(nuevos_beats) * 0.5  
        X_replay, y_replay = replay_buffer.sample(n_replay)  
  
        # Paso 3: Combinar nuevos datos + datos del buffer  
        X_combinado = concat(nuevos_beats, X_replay)  
        y_combinado = concat(nuevas_labels, y_replay)  
  
        # Paso 4: Mezclar aleatoriamente  
        permutacion_aleatoria(X_combinado, y_combinado)  
  
        # Paso 5: Backpropagation con lr=0.0001  
        for batch in DataLoader(X_combinado, y_combinado):  
            loss = CrossEntropyLoss(model(batch), labels)  
            loss.backward()  
            optimizer.step()  
  
        # Paso 6: Guardar modelo actualizado  
        torch.save(model.state_dict(), "ADAPT-ECG-RETRAINED.pth")  
  
        # Paso 7: Persistir el buffer para la próxima sesión  
        replay_buffer.save("replay_buffer.npz")
```

8.4 Historial Real de Reentrenamiento (5 Sesiones)

Sesión	Fecha	Registro	Latidos	Acc. Antes	Acc. Después	F1 Antes	F1 Después
1	2026-03-18 19:10	100	2,272	98.55%	99.08%	66.42%	84.29%
2	2026-03-18 19:10	103	2,084	77.69%	99.90%	29.29%	49.98%
3	2026-03-18 22:44	112	2,539	87.00%	99.92%	18.61%	49.98%
4	2026-03-18 22:45	100	2,272	98.55%	99.34%	66.42%	91.12%
5	2026-03-18 22:46	124	88	81.82%	73.86%	36.14%	33.66%

Nota sesión 5: La degradación en el registro 124 se debe a que solo se usaron **88 latidos** — muy pocos para que el reentrenamiento sea estable. Con datasets pequeños, la actualización puede ser ruidosa.

8.5 Resultados Fase 3 (Estático vs Adaptativo)

Métrica	Modelo Estático	Modelo Adaptativo	Mejora
Accuracy	93.54%	97.83%	+4.29%
F1-macro	78.49%	90.71%	+12.22%
F1-weighted	94.47%	97.73%	+3.26%

9. Evaluación Final (Fase 4)

9.1 Métricas Globales sobre 75,702 latidos

Métrica	Modelo Estático	Modelo Adaptativo	Mejora
Accuracy	88.72%	97.41%	+8.69%
F1-macro	73.81%	89.54%	+15.73%
F1-weighted	91.14%	97.36%	+6.22%
Precision-macro	68.91%	92.55%	+23.64%
Recall-macro	92.48%	86.99%	-5.49%

9.2 Métricas por Clase

Modelo Estático

Clase	Precisión	Sensibilidad	Especificidad	F1
N (Normal)	99.32%	87.11%	97.72%	92.81%
S (Supraventricular)	26.82%	91.01%	91.74%	41.43%

Clase	Precisión	Sensibilidad	Especificidad	F1
V (Ventricular)	92.27%	93.21%	99.36%	92.74%
F (Fusión)	29.53%	93.22%	98.08%	44.85%
Q (Desconocido)	96.61%	97.86%	99.66%	97.23%

Modelo Adaptativo

Clase	Precisión	Sensibilidad	Especificidad	F1
N (Normal)	98.11%	99.05%	92.68%	98.58%
S (Supraventricular)	81.32%	73.78%	99.44%	77.37%
V (Ventricular)	96.16%	93.28%	99.69%	94.70%
F (Fusión)	89.06%	71.49%	99.92%	79.32%
Q (Desconocido)	98.12%	97.36%	99.82%	97.74%

La mayor mejora es en **Fusión (F)**: precisión de 29.53% → 89.06%. El modelo estático tenía muchos falsos positivos en esta clase difícil.

9.3 AUC-ROC (Área bajo la curva ROC)

Clase	Estático	Adaptativo
N	0.9909	0.9913
S	0.9743	0.9740
V	0.9980	0.9989
F	0.9930	0.9939
Q	0.9996	0.9997

AUC > 0.97 en todas las clases — el modelo es un discriminador excelente.

9.4 Test Estadístico de McNemar

Se aplicó el **test de McNemar** para verificar que la diferencia entre modelos no es por azar:

Resultado	Valor
Estadístico	5,342.29
p-valor	0.0000
¿Significativo?	Sí ($p < 0.05$)

La mejora del modelo adaptativo sobre el estático es **estadísticamente significativa**. No es coincidencia.

10. Interfaz de Usuario (Streamlit)

Ejecutar con: `streamlit run src/ui/app.py`

Modo 1 — MIT-BIH:

1. Selecciona una carpeta con registros MIT-BIH
2. Elige un registro (100–234)
3. La app carga la señal, filtra, segmenta y clasifica cada latido
4. Muestra el ECG con los picos R coloreados por clase
5. Muestra gráfica de distribución de clases y métricas
6. Botón para **reentrenar el modelo** con ese registro

Modo 2 — CSV propio:

1. Sube un archivo CSV con una columna de voltaje en mV
2. Especifica la frecuencia de muestreo
3. La app detecta picos R automáticamente con `scipy.signal.find_peaks`
4. Clasifica cada latido detectado

11. Stack Tecnológico

Librería	Versión	Uso
PyTorch	≥2.0	Definición y entrenamiento de la CNN
Streamlit	≥1.30	Interfaz web interactiva
wfdb	≥4.1	Lectura de registros MIT-BIH (.dat, .hea, .atr)
NumPy	≥1.24	Operaciones numéricas y arrays
SciPy	≥1.11	Filtro Butterworth, detección de picos
scikit-learn	≥1.3	Métricas de evaluación
Plotly	≥5.18	Gráficas interactivas
Matplotlib / Seaborn	≥3.7	Matriz de confusión
Pandas	≥2.0	Manejo tabular de datos

12. Estructura Final del Proyecto

```
PROJECT_ADAPT-ECG/
├── src/
│   ├── __init__.py
│   ├── config/
│   │   └── settings.py      ← parámetros globales
│   ├── data/
│   │   ├── ingest.py       ← descarga y carga MIT-BIH
│   │   └── preprocess.py   ← filtrado, segmentación, normalización
│   └── ui/
│       └── app.py          ← interfaz Streamlit completa
└── notebooks/
```

- ■■■ fase2_entrenamiento.ipynb
- ■■■ fase3_reentrenamiento.ipynb
- ■■■ fase4_evaluacion.ipynb
- models/
 - ■■■ ecg_cnn_base.pth ← modelo estático
 - ■■■ ecg_cnn_base_meta.json
 - ■■■ ADAPT-ECG-RETRAINED.pth ← modelo adaptativo
 - ■■■ replay_buffer.npz ← memoria del buffer
 - ■■■ retrain_history.json ← historial de sesiones
 - ■■■ fase3_resultados.json
 - ■■■ fase4_resultados.json
- data/
 - ■■■ raw/ ← 48 registros MIT-BIH descargados
 - ■■■ processed/
 - ■■■ X.npy ← latidos preprocesados (94,627 × 72)
 - ■■■ y.npy ← etiquetas numéricas
- docs/
 - ■■■ fase1_verificacion.png
 - ■■■ fase2_latidos_por_clase.png
 - ■■■ fase3_comparacion.png
 - ■■■ fase4_confusion_matrices.png
 - ■■■ fase4_evolucion_lotes.png
 - ■■■ fase4_metricas_por_clase.png
 - ■■■ fase4_roc_curves.png
- scripts/
 - ■■■ run_phase1.py
 - ■■■ retrain.ps1
 - ■■■ train.ps1
 - ■■■ run_api.ps1
- requirements.txt
- README.md

13. Conclusiones

1. **La CNN 1D entrenada con MIT-BIH alcanza 93.18% de accuracy** en validación, superando el umbral mínimo clínico para sistemas de apoyo al diagnóstico.
2. **El Replay Buffer resuelve el olvido catastrófico** al mezclar datos históricos con datos nuevos en cada sesión de reentrenamiento.
3. **El modelo adaptativo supera estadísticamente al estático** ($p=0.0$ en McNemar), con mejoras de hasta +15.7% en F1-macro.
4. **La clase más difícil es Fusión (F)** por su baja prevalencia (0.9%) y su naturaleza ambigua, pero el modelo adaptativo logra mejorarla de $F1=44.85\%$ a $F1=79.32\%$.
5. **El sistema es funcional en tiempo real** a través de la interfaz Streamlit, permitiendo inferencia y reentrenamiento desde la UI sin necesidad de línea de comandos.